LLM heuristics benchmarking: team plan

Oct 26, 2026 · @Shouvik

Four people, one month: test how robust LLM-generated planning heuristics really are, building on the released code of Corrêa et al. (NeurIPS 2025).

Proposal (submit today)

> **Title:** How robust are LLM-generated heuristics for classical planning?
>
> **Option:** Benchmarking. **Topic:** Planning and foundation models.
>
> Corrêa et al. (NeurIPS 2025) show that LLMs can write domain-specific Python heuristics that, inside greedy best-first search, solve more IPC 2023 Learning Track tasks than h_FF. We benchmark how robust this result is along three axes the original study leaves mostly open. (1) **Semantic cues:** we systematically obfuscate all eight domains at increasing levels (names only, then names plus object types, then reordered PDDL) and measure the drop in coverage. (2) **Model scale:** we compare frontier models against small open-weight models that can run on a laptop. (3) **Selection:** using the 25 archived candidate heuristics per domain, we test whether better selection rules or simple portfolios close the gap between the selected heuristic and the best available one. Baselines are Pyperplan with h_FF and blind search, and Fast Downward with h_FF and LAMA. We report coverage, node expansions, runtime and plan length, with figures for coverage versus obfuscation level, coverage versus model size, and expansions of LLM heuristics versus h_FF. All code and data build on the authors' public release (Zenodo, CC BY 4.0).

Adjust the model names in axis 2 once you know your API budget and hardware (see the last section).

What the paper already tested

The original paper is more thorough than I said in my first notes. It already ran a sample-size study, prompt ablations and a one-domain obfuscation test, so our project has to go beyond those rather than repeat them.

Check in the paper	What they found	What is still open (our angle)
Obfuscation: all Blocksworld symbols renamed	DeepSeek R1 heuristic drops from 66 to 40 solved; h_FF barely changes (27 to 28)	Only one domain and one model. No graded levels. **Axis 1**
Novel domain Rod-Rings (never online)	o3 heuristic solves 58, close to Fast Downward h_FF at 59	One domain, one model
Modified Spanner (spanners never break)	Best heuristic is perfect	Not a benchmark, an anecdote
Number of candidates n (Gemini 2.0 Flash)	Biggest gain from 1 to 5; n = 25 solves all training tasks in 6 of 8 domains	Settled; we just use n = 10 to save compute
Prompt ablations (Flash Thinking)	Every component helps; removing the checklist hurts most	Settled; we keep the full prompt
Smaller models	R1-Distill-Qwen-14B (214) is below h_FF (243)	Authors list it as an open question. **Axis 2**
Selection rule	Best training coverage; the best ablation reached 423 while the selected one was lower	Authors list better selection as future work. **Axis 3**
No feedback loop to the LLM	Not tried	Authors suggest it as future work. **Stretch goal**

The released archive (Zenodo, CC BY 4.0) contains the generation script, Pyperplan integration, every prompt and LLM answer, all generated heuristics including ablations, the IPC 2023 training and test splits, and the experiment scripts. That removes most of the setup risk.

Research questions

One umbrella question: **when LLM heuristics beat h_FF, is it because of what the model reasons from the PDDL, which model you use, or which candidate you happen to pick?** Each axis isolates one of those factors.

Axis 1: semantic cues (obfuscation)

RQ1. How much of an LLM heuristic's coverage depends on meaningful names in the PDDL?

Four levels, applied to all 8 domains with the same renaming in domain and task files:

1. **L0 original.**
2. **L1 renamed:** predicates, actions and types become random strings (this replicates the paper's Blocksworld test).
3. **L2 renamed + shuffled:** L1 plus object names randomised and the order of predicates, actions, parameters and initial facts permuted.
4. **L3 misleading:** names consistently swapped for meaningful words from a different domain (Blocksworld's `on` becomes `in-truck`). This tests whether wrong cues actively hurt, not just whether missing cues do.

Hypotheses. Coverage falls from L0 to L2; the fall is largest in famous domains (Blocksworld, Miconic, Sokoban) and smallest in less-known ones; L3 hurts more than L1. h_FF coverage must not change across levels, which doubles as a correctness check on the obfuscator.

Axis 2: model scale

RQ2. How small can the model be before its heuristics fall below h_FF, and does a larger sample n compensate for a smaller model?

Compare DeepSeek V3 and R1 (results already archived, so free to re-use) against 2-3 sizes of one open-weight model family, e.g. coder models around 7B, 14B and 32B, run locally or through a cheap OpenAI-compatible endpoint. For the smallest model also try n = 25 vs. n = 10.

Hypotheses. Coverage rises with size; below some size most candidates fail to run at all (crashes, infinite values), so the failure rate is as informative as coverage.

Axis 3: selection and combination

RQ3. Is "most training tasks solved" a good way to pick a heuristic, and can a small portfolio beat any single pick?

Using the archived 25 candidates per domain (R1), evaluate every candidate on a test subset, then compare selection rules after the fact: the paper's rule; best on the largest training tasks only; fewest training expansions; and the oracle (best on test) as an upper

bound. Then implement **alternation**, LAMA's trick of round-robin over several open lists (lecture 7), over the top 2-3 candidates in Pyperplan.

Hypotheses. The paper's rule is far from the oracle in at least 2-3 domains; weighting large training tasks closes part of the gap; alternation beats the single best pick, as it does for LAMA.

Stretch: one round of feedback

Feed the selected heuristic back to the LLM with its training failures (unsolved tasks, expansion counts) and ask for an improved version. Only if axes 1-3 are on schedule by Nov 9.

Shared protocol

Everyone runs experiments the same way, so results from four laptops can be merged into one table.

Setting	Value	Why
Code base	The authors' Zenodo release, in a shared Git repo (private GitHub)	Same prompt, same Pyperplan, comparable to the paper
Domains	Blocksworld, Childsnack, Floortile, Miconic, Rovers, Sokoban, Spanner, Transport	The paper's 8 IPC 2023 Learning Track domains
Test subset	30 of the 90 test tasks per domain (every third, so all sizes are covered); 240 tasks	Fits the compute budget, keeps the size range
Training for selection	The paper's training tasks, 60 s per run	The paper used 5 min; shorter keeps selection cheap
Candidates per configuration	n = 10	The paper shows most of the gain comes by n = 5
Search	GBFS in Pyperplan under PyPy	Same as the paper
Test limits	5 min, 4 GiB per task, one core	Paper used 30 min / 8 GiB; state the change clearly in the report

Setting	Value	Why
Seeds	3 independent generation runs per LLM configuration	LLM sampling varies; report mean and range
Pyperplan baselines	Blind, h_FF	Same slow engine: fair test of heuristic quality
Fast Downward baselines	GBFS + h_FF (eager), LAMA-2011 first iteration	The real state of the art, for context

Metrics, logged for every run: solved (yes/no), wall time, node expansions, plan length (validated with VAL), peak memory, and for each generated candidate whether it crashed, returned infinity, or timed out. Coverage, the IPC agile score and the candidate failure rate are computed from these.

One results format. Every run writes one JSON line: `{domain, task, config, model, obf_level, seed, candidate_id, solved, time, expansions, plan_len, mem, error}`. Member 4's plotting scripts only read this format, so nobody hand-edits tables.

Comparability rule. Re-run the paper's selected R1 heuristic and Pyperplan h_FF under our 5 min limit first. Every other number is compared against those re-runs, never against the paper's 30 min figures.

Roles

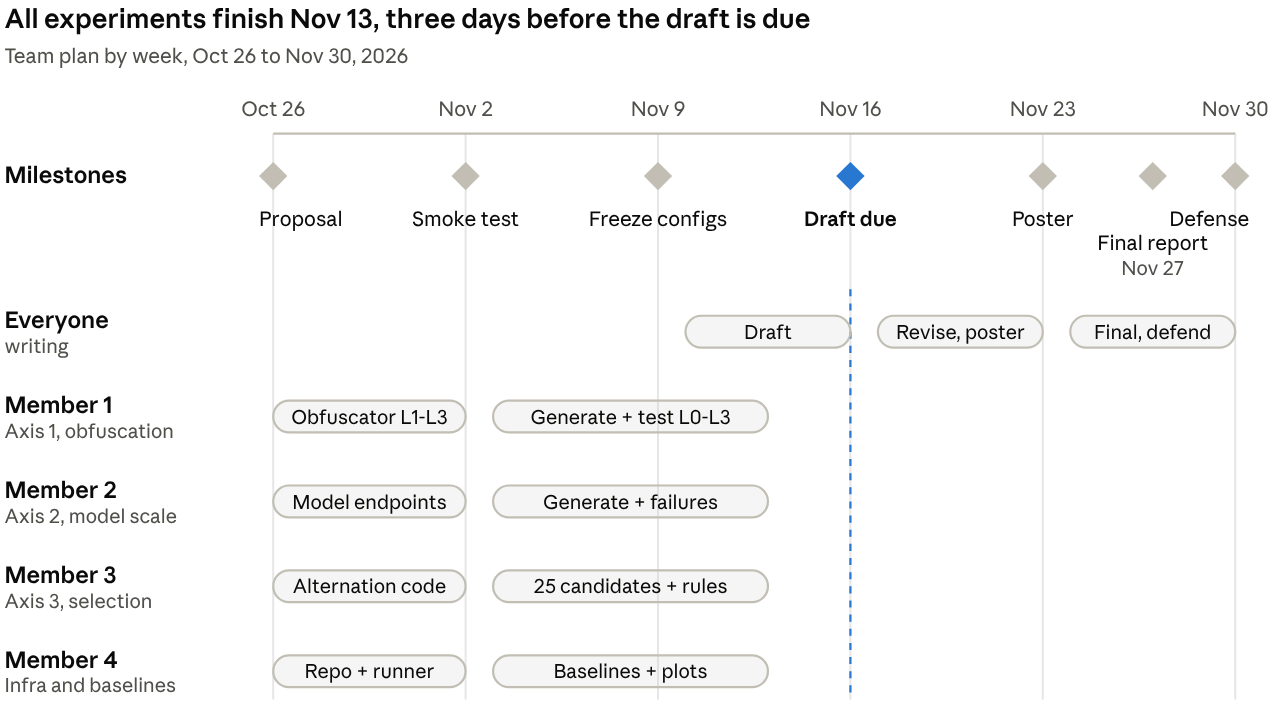
Each person owns one axis end to end, from code to figure to paper section, and Member 4 owns the shared infrastructure that the other three depend on. Replace "Member N" with names.

Member	Owns	Builds	Runs	Writes (report)	Poster panel
Member 1	Axis 1: obfuscation	`obfuscate.py` producing L1, L2 and L3 versions of all 8 domains and their tasks	Heuristic generation on L1-L3 (R1 and V3, n = 10, 3 seeds); tests on 240 tasks; h_FF invariance check	Axis 1 method and results; related work on LLM memorisation	"Do names matter?"

Member	Owns	Builds	Runs	Writes (report)	Poster panel
Member 2	Axis 2: model scale	OpenAI-compatible endpoint support in `llm-heuristics.py` (local server or cheap API)	Generation with 2-3 open model sizes; failure taxonomy of every candidate (crash, infinite h, timeout, works)	Axis 2 method and results; related work on LLMs for planning	"How small can the model be?"
Member 3	Axis 3: selection	Alternation (round-robin open lists) in Pyperplan's GBFS	All 25 archived R1 candidates on the test subset; selection rules; alternation over top 2-3	Axis 3 method and results; background on heuristic search and LAMA	"Picking the right heuristic"
Member 4	Infrastructure, baselines, integration	Shared repo, parallel run script, results schema, plotting scripts, LaTeX template	Pyperplan blind and h_FF; Fast Downward h_FF and LAMA; re-runs of the paper's R1 heuristics	Experimental setup, introduction, conclusion; final editing pass	Title, setup and headline result

Member 4 is also the representative who talks to the professor. Everyone reads the original paper fully in week 1; each member also reads the 2-3 papers behind their related-work part.

Timeline



team timeline · Oct 26 to Nov 30

Week 1 is building, week 2 is running, and writing starts while the last runs finish. Nothing new gets added after the Nov 9 freeze, so the Nov 16 draft has complete results.

Week 1 tasks for everyone besides their build: read the paper in full, download the Zenodo archive, and reproduce one domain (Blocksworld, R1 heuristic, Pyperplan) on your own machine by Nov 2.

Figures, paper and poster

The brief requires at least two figures with well-defined metrics; plan five, so the weakest can be cut.

#	Figure	Shows	Owner
F1	Coverage per domain at L0-L3, grouped bars, h_FF as a marker per domain	Where names matter and where they don't	Member 1

#	Figure	Shows	Owner
F2	Coverage vs. model size (log scale), h_FF as a horizontal line; candidate failure rates as a stacked bar beneath	The size at which LLM heuristics stop paying off	Member 2
F3	Per domain, test coverage of all 25 candidates as dots, with the paper's pick, our rules, the oracle and alternation marked	How much selection leaves on the table	Member 3
F4	Expansions, LLM heuristic vs. h_FF, log-log scatter with the diagonal	Is the gain better guidance or something else?	Member 4
F5	Cactus plot: tasks solved vs. time, every main configuration	One-glance summary for the poster	Member 4

Report outline (aim for 6-8 pages, two-column, ICAPS LaTeX style).

1. Introduction: the claim being stress-tested and our three axes (Member 4).
2. Background: STRIPS, GBFS, h_FF, LAMA and alternation (Member 3).
3. Related work: LLMs as planners vs. LLMs writing planning code; memorisation and contamination (Members 1 and 2).
4. Experimental setup: the shared protocol table (Member 4).
5. Results: one subsection per axis (Members 1-3).
6. Discussion and limitations: shorter time limits than the paper, Python planner, one model family for scaling (all).
7. Conclusion and future work (Member 4).

Poster. Title and the one-sentence answer at the top; four panels underneath matching the roles table; F5 in the centre as the summary. Each member presents their own panel at the Nov 30 defense.

Compute, cost and risks

The whole plan needs roughly 600 CPU-hours and under $40 of API calls. That is about 25 hours of wall time on 4 laptops with 6 free cores each, so it fits, but ask the professor about cluster access in week 1 anyway.

These are my estimates, assuming about half of test runs time out at 5 min and training runs mostly finish quickly; check them against your smoke tests on Nov 2.

Workload	Configurations	Estimate (CPU-hours)
Baselines and re-runs	4	~40
Axis 1: selection + tests	3 levels × (R1 × 3 seeds + V3 × 1) = 12	~230
Axis 2: selection + tests	3 sizes × 3 seeds = 9	~170
Axis 3: all 25 candidates on 15 tasks per domain, plus alternation	25 + 3	~150

API cost: the paper spent $6.12 on R1 for 8 domains at n = 25, so n = 10 is about $2.50 per full generation. Axis 1 needs about 9 of those with R1 plus cheap V3 runs; prices may have changed since the paper.

Risks and fallbacks.

- **Obfuscator bugs** would invalidate Axis 1. Guard: h_FF coverage must be identical across levels, and every plan is validated with VAL against the obfuscated domain.
- **Compute overrun.** Cut in this order: the stretch goal, then L3, then test tasks from 30 to 20 per domain.
- **API model or price changes.** Use whichever R1-class reasoning model is available, record the exact version and date, and re-run its L0 so comparisons stay within one model.
- **Small models produce no working heuristic.** That is a valid result for Axis 2; the failure taxonomy becomes the figure.
- **Pyperplan rejects shuffled PDDL.** Test L2 on one domain on day one of Member 1′s work.

Decision gates. Nov 2: every pipeline runs end to end on one domain. Nov 9: experiment configurations frozen, no new ideas after this. Nov 14: all runs finished, figures drafted.

Open questions for the group.

- Who takes which role?
- Do we have cluster access, or only laptops?
- Who pays for the API calls, and is $40 acceptable?

- Does the professor accept 5 min limits instead of the paper's 30 min, if we state it clearly?

Sources: Corrêa et al. 2025, arXiv, full paper PDF, code and data on Zenodo, NeurIPS 2025 slides.